Docker Implementation Report

Application of Choice: Campus Resource Booking System (CRBS)

Student Name:	Mugunth	**Live App URL:**	campus-resource-booking-cqt7.onrender.com
Department:	Computer Science & Engineering	**GitHub Repo:**	github.com/Mugu1825/Campus-Resource-Booking
Container Tech:	Docker, Nginx, Gunicorn, Python 3.12-slim	**Cloud Runtime:**	Render Web Service (Docker Environment)

1. Introduction & Why Docker?

Deploying full-stack applications with separate frontend static assets, Python WSGI backends, and cloud databases typically requires configuring multiple independent servers or struggling with Cross-Origin Resource Sharing (CORS). Furthermore, differences in local machine packages frequently cause unexpected deployment bugs.

By implementing **Docker containerization** for the Campus Resource Booking System, the complete runtime environment—operating system libraries (Debian), Python interpreter, dependencies, Gunicorn WSGI gateway, and Nginx web server—is compiled into an immutable image. The app runs identically on any developer machine and in the cloud with zero configuration friction.

2. Container Architecture: Single-Container Model

I designed a **single-container architecture** combining Nginx and Gunicorn. Nginx listens on port 80 (or Render's dynamic $PORT), directly serves all static frontend assets (HTML/CSS/JS) with caching, and transparently reverse-proxies all /api/* requests to Gunicorn on 127.0.0.1:5000. This architecture guarantees:

• **Zero CORS Issues:** Frontend and backend are accessed on the exact same origin and port.
• **Cost Efficiency:** Packaged into a single web service instead of paying for or managing separate instances.
• **Dynamic Cloud Port Binding:** At startup, an entrypoint script reads Render's injected $PORT variable and configures Nginx dynamically.

```
[Browser Request (Port 80 / $PORT)] --> [Nginx Reverse Proxy in Docker Container] |--> Static Assets (/*, /css/*, /js/*) --> Served from /usr/share/nginx/html |--> API Traffic (/api/*) -------------> Proxied to Gunicorn on 127.0.0.1:5000 | v [TLS 1.3 Encryption] [MongoDB Atlas Cloud Database]
```

3. Breakdown of Docker Configuration Files

File	Key Directives & Configuration	Purpose in Container Lifecycle
Dockerfile	FROM python:3.12-slim RUN apt-get update && apt-get install -y nginx curl COPY backend/requirements.txt ... RUN pip install --no-cache-dir -r ... COPY frontend /usr/share/nginx/html ENTRYPOINT ["/app/entrypoint.sh"]	Multi-tier image definition. Installs Nginx and Python runtime, installs PyMongo/Flask/Bcrypt, copies frontend assets to Nginx standard directory, and defines entrypoint.
nginx.conf	location / { try_files $uri $uri/ /index.html; } location /api/ { proxy_pass http://127.0.0.1:5000/api/; ... } location /css/, /js/ { expires 1d; }	Web server configuration. Handles static file delivery, gzip compression, client-side routing, and reverse-proxying API calls to Gunicorn with proxy headers.
entrypoint.sh	sed -i "s/LISTEN_PORT/${PORT}/g" default.conf gunicorn --bind 127.0.0.1:5000 app:app & curl http://127.0.0.1:5000/api/health exec nginx -g 'daemon off;'	Container orchestrator script. Binds Nginx to Render's dynamic $PORT, spawns Gunicorn in the background, waits for health check, and runs Nginx in the foreground.
docker-compose.yml	version: '3.8' services: app: build: . ports: ["3000:80"] env_file: [./backend/.env]	One-command local development setup. Maps host port 3000 to container port 80 and passes Atlas credentials securely from local environment files.

4. Step-by-Step Execution & Commands

Step 1 — Build the Docker Image:
docker build -t campus-booking .
Docker compiles the base image, packages Nginx, installs Python wheels, and copies application code.
Step 2 — Run the Container Locally:
docker run -d --name campus-booking-app -p 3000:80 --env-file backend/.env campus-booking
Or alternatively using Docker Compose: docker compose up -d
Step 3 — Verify Running Status & Logs:
docker ps *(shows container active on port 3000)*
docker logs campus-booking-app *(shows Gunicorn healthy and Nginx ready)*
Step 4 — Test in Browser:
Open http://localhost:3000 in your browser to interact with the containerized application.

5. Cloud Deployment via Docker on Render

For cloud production, the repository was linked to **Render Cloud** as a **Docker Web Service**. Render's automated CI/CD pipeline detected the Dockerfile, compiled the container in the cloud, supplied the MONGO_URI and JWT_SECRET environment variables, and deployed the live service at: **https://campus-resource-booking-cqt7.onrender.com**.

6. Conclusion & Benefits Realized

Containerizing the Campus Resource Booking System with Docker achieved a fully reproducible, cloud-ready deployment. It eliminated cross-origin resource sharing complexities through an internal Nginx reverse proxy, simplified multi-tier process management with an automated entrypoint script, and allowed one-click deployment to cloud platforms.